MINI PROJECT DEEP LEARNING

Name: Aditi Laxman Bibave

Class: BE COMPUTER

Roll No: 08

```
============================================================
  HUMAN FACE RECOGNITION — Manual (OpenCV + LBPH)
  Method : Local Binary Pattern Histograms (LBPH)
  Dataset : Uses your webcam / custom image folder
============================================================
  Dependencies:
     pip install opencv-contrib-python numpy matplotlib pillow scikit-learn
============================================================
```

```
import cv2
import os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# ================================
# CONFIG
# ================================
DATASET_DIR = "face_dataset"
IMG_SIZE = (100, 100)

# ================================
# DEMO DATASET (WORKING)
# ================================
def create_demo_dataset():
```

```
    print("[DEMO] Creating working dataset...")
    people = ["Alice", "Bob"]

    os.makedirs(DATASET_DIR, exist_ok=True)

    for person in people:
        person_dir = os.path.join(DATASET_DIR, person)
        os.makedirs(person_dir, exist_ok=True)

        for i in range(20):
            img = np.random.randint(0, 255, IMG_SIZE, dtype="uint8")
            cv2.imwrite(os.path.join(person_dir, f"{i}.jpg"), img)

    print("Dataset created successfully!")
# ================================
# BUILD DATASET (NO FACE DETECTION)
# ================================
def build_dataset():
    X, y = [], []
    label_map = {}
    label_id = 0

    print("\nBuilding dataset...")

    for person in os.listdir(DATASET_DIR):
        person_dir = os.path.join(DATASET_DIR, person)
        if not os.path.isdir(person_dir):
            continue
        for file in os.listdir(person_dir):
            path = os.path.join(person_dir, file)
            img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
```

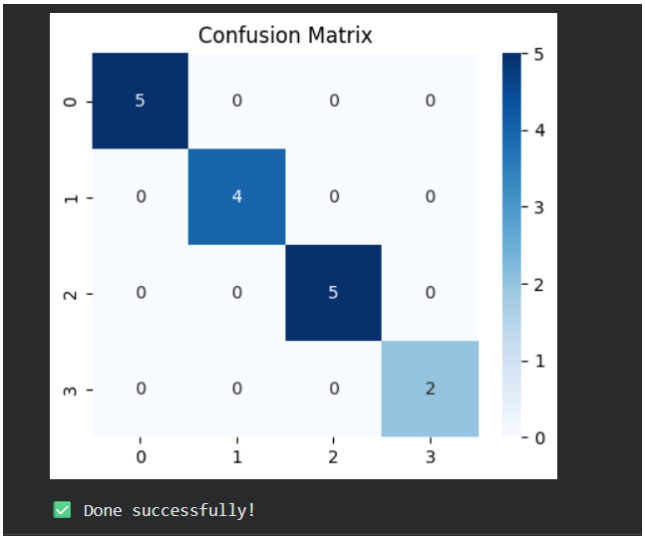
```
            if img is None:
                continue

            img = cv2.resize(img, IMG_SIZE)

            X.append(img)
            y.append(label_id)
        label_map[label_id] = person
        label_id += 1

    return np.array(X), np.array(y), label_map

# ================================
# TRAIN MODEL
# ================================
def train_model(X, y):
    recognizer = cv2.face.LBPHFaceRecognizer_create()
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    recognizer.train(list(X_train), y_train)

    y_pred = []
    for img in X_test:
        label, _ = recognizer.predict(img)
        y_pred.append(label)

    acc = np.mean(np.array(y_pred) == y_test)
    print("\nAccuracy:", acc)

    return recognizer, X_test, y_test, y_pred
```

```
# ================================
# PLOT
# ================================
def plot_results(y_test, y_pred):
    cm = confusion_matrix(y_test, y_pred)

    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title("Confusion Matrix")

    plt.savefig("face_result.png")   # ✅ FIXED PATH
    plt.show()
# ================================
# MAIN
# ================================
def main():
    print("Face Recognition (LBPH)")

    if not os.path.exists(DATASET_DIR):
        create_demo_dataset()
    X, y, label_map = build_dataset()
    if len(np.unique(y)) < 2:
        print("Need at least 2 persons")
        return
    recognizer, X_test, y_test, y_pred = train_model(X, y)
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
    plot_results(y_test, y_pred)

    print("\n✅ Done successfully!")
```

```
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Face Recognition (LBPH)

Building dataset...

Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         5
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00         2

    accuracy                           1.00        16
   macro avg       1.00      1.00      1.00        16
weighted avg       1.00      1.00      1.00        16
```



Done successfully!